HotelEASE(Easy Access to a Simple and Effective Hotel Management System)

A Project
Presented to the
Faculty of the College of Information Technology and Education
PHINMA ST. JUDE COLLEGE
Sampaloc Manila

In Partial Fulfillment
of the Requirements in
OBJECT ORIENTED PROGRAMMING

By:

Barros, Dezire D.
Cailo, Johnrey C.
Canlas, Danilo Jr B.
Larong, Jheboy
Simon, Jericho Martin

October 2024

ABSTRACT

HotelEASE is a package and comprehensive hotel management and payroll system designed to maximize the efficiency of hotels in terms of running the establishment and enhancing administrative efficiency. The general objective is to automate essential functions that include room booking, guest management, as well as payroll processing, thereby bettering the resource management capability of the hotel administrator.

The hotel management module has features including room availability tracking, reservation, billings, and customer services. Part of the payroll module under the hotel management module is the key dealing with staff salaries and time, attendance, and taxes computation for accurate and timely payroll.

By combining such functions into a single system, HotelEASE reduces redundant tasks and reduces the likelihood of human errors, thus improving operational efficiency. The system also provides a friendly user interface that ensures ease in use for both hotel staff and administrators while providing comprehensive reports and analytics to support the decision-making process.

HotelEASE is targeting bringing hotels modern, scalable solutions that make a big difference in guest satisfaction and employee management to maximize production and profits.

TABLE OF CONTENTS

I. INTRODUCTION

I.I Overview of the System 1

I.II Background of the System 2

I.III Objective of the System 2

II. SYSTEM ARCHITECTURE

II.I Use Case Diagram (UML) 4

II.II Flowchart 7

II.III Data Flow Diagram 9

II.IV Entity Relationship Diagram 10

III. DEVELOPMENT TOOLS AND ENVIRONMENT

III.I Programming Language 11

III.II Development Tools 11

III.III Methodology 11

IV. KEY SYSTEM COMPONENTS

IV.I Attributes 12

V. ADVANTAGES OF USING OOP IN THE SYSTEM

V.I Modularity 15

V.II Flexibility 15

VI. POTENTIAL CHALLENGESS AND SOLUTIONS

VI.I Challenges 15

VI.II Mitigation Strategies 16

VII. TESTING AND IMPLEMENTATION PLAN

VII.I Unit Testing 16

VII.II Integration Testing 16

VII.III Deployment Strategy 16

VIII. CONCLUSION

VIII.I Summary 17

VIII.II Future Enhancements 17

IX. APPENDICES

IX.I Appendix A: Evidences of Collaboration 17

IX.II Appendix B: Results of Unit Testing and Integration Testing 19

IX.III APPENDIX C: CURRICULUM VITAE

I. INTRODUCTION

I.I Overview of the System:

Introduction: HotelEASE is a comprehensive hotel management system to streamline and automate various aspects of hotel operations, providing a seamless experience for both hotel staff and guests. This system integrates key hotel functions such as room reservations, front desk management, housekeeping, billing, and guest services into one unified platform. Also included in this is the Payroll System of the workers of those who work at the hotel itself

Problem the System is Addressing: Hotels often face inefficiencies due to the manual management of operations such as room bookings, check-ins/outs, staff coordination, and billing processes. These inefficiencies can lead to errors, time delays, customer dissatisfaction, and revenue losses. HotelEASE addresses these challenges by automating routine tasks, reducing the likelihood of human error, improving operational efficiency, and enhancing the guest experience.

Main Purpose: The main purpose of HotelEASE is to provide a centralized system that simplifies hotel management. It enables staff to manage reservations, assign rooms, track occupancy rates, handle guest requests, and monitor housekeeping duties in real-time. Additionally, it supports financial operations by automating invoicing, payments, and reporting, ultimately increasing profitability and improving overall hotel service quality.

I.III Background of the Study/System

In today's highly competitive hospitality industry, hotels face growing pressure to deliver exceptional guest experiences while optimizing operational costs. Traditional management methods, often reliant on manual processes or legacy software, are becoming insufficient to meet the increasing expectations of both hotel guests and management teams. The need for a modern, integrated solution has led to the development of comprehensive hotel management systems. These systems bring together various functions such as reservations, billing, room management, and staff coordination into a single platform, enabling hotels to run efficiently while providing a superior guest experience. **HotelEASE** is proposed to address the challenges faced by hotels in managing these core functions. The system will leverage Object-Oriented Programming (OOP) principles to create a modular, flexible, and scalable solution that can grow with the hotel's needs and adapt to various types of properties, from boutique hotels to large resorts.

I.II Objective of the Proposal:

Goals of the Proposal: The proposal aims to develop HotelEASE, a hotel management system designed to enhance operational efficiency, improve guest satisfaction, and optimize resource utilization. The specific goals include:

- Streamline Hotel Operations: Automate and integrate various hotel functions such as reservations, room assignments, billing, housekeeping, and customer service.
- Enhance Guest Experience: Provide seamless booking experiences, quick check-ins/outs, and effective communication channels between guests and hotel staff.

- Improve Financial Management: Automate payment processing, invoicing, and generate financial reports to optimize revenue tracking and cash flow management.
- Increase Staff Productivity: Centralize operational information, making it easy for staff to coordinate tasks and manage hotel operations efficiently.
- Data Management and Analytics: Enable hotel management to track performance metrics, occupancy rates, and customer preferences through data collection and analysis for better decision-making.

Why Object-Oriented Programming (OOP) is a Suitable Approach: OOP is well-suited for developing HotelEASE due to its modular and flexible structure, which aligns with the complex nature of hotel management systems. Here's why:

- **Encapsulation**: OOP enables bundling of data (e.g., guest details, room status, billing) and the functions that manipulate that data into objects, allowing clear structure and better data management.
- **Reusability**: With classes representing entities such as Guests, Rooms, Reservations, and Staff, OOP promotes code reusability and reduces redundancy, making the system easier to maintain and expand.
- **Inheritance**: OOP allows new features to be added by building on existing classes. For instance, special types of rooms or services (e.g., suites or VIP services) can be created by extending base classes.
- **Polymorphism**: The system can handle different types of tasks (e.g., different payment methods, customer preferences) through polymorphism, providing flexibility in implementation.

II System Architecture

II.I Use Case Diagram

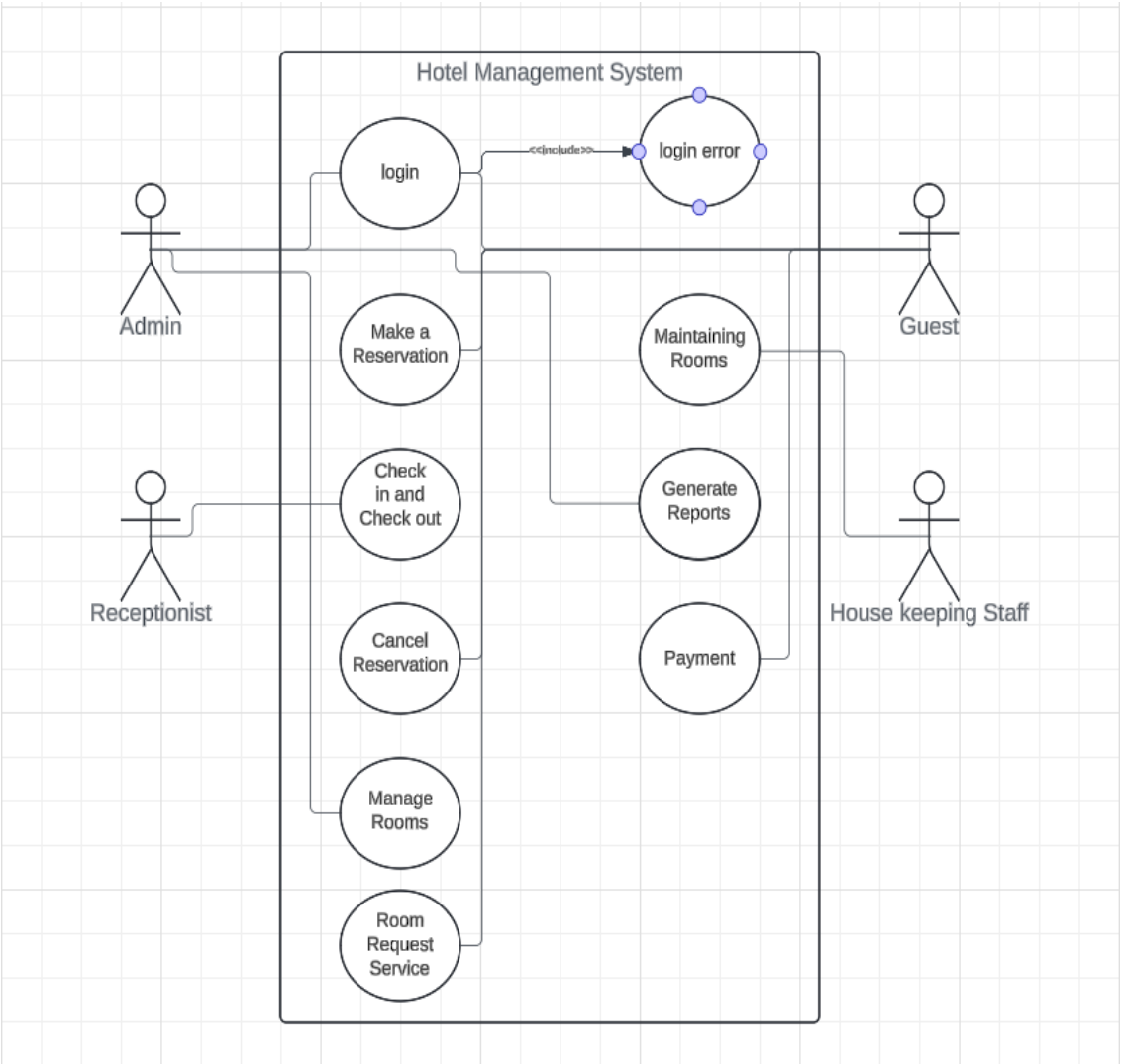

Fig 2.1 use case diagram

Actors:

- **Admin:** Responsible for managing the overall hotel management system. They typically handle user access, generate reports, and monitor system operations.

- **Receptionist:** Handles customer-facing tasks such as making reservations, managing check-ins and check-outs, and handling payments.
- **Guest**: Can make reservations, request services, check in/out, and potentially interact with the payment process.
- **Housekeeping Staff:** Responsible for maintaining rooms and ensuring they are prepared and clean for guests.

Use Cases:

- **Login**: All actors (Admin, Receptionist, Housekeeping Staff, Guest) must log in to access their specific functionality. The diagram shows a system response for an error if the login fails (indicated by the "login error" bubble).
- **Make a Reservation**: The Receptionist and Guest can use this functionality to book rooms. This is a core function for any hotel management system.
- **Check-in and Check-out**: Guests and Receptionists handle check-ins and check-outs. The receptionist assists with this process, and it could be automated for guests via a self-service feature.
- **Cancel Reservation**: Guests and Receptionists have the ability to cancel room reservations.
- **Manage Rooms**: Receptionists or housekeeping staff can manage room availability, status (clean, occupied, etc.), and perform other related tasks.
- **Room Request Service**: Guests can request services related to their room (e.g., room cleaning, amenities requests) through the system.
- **Maintaining Rooms**: Housekeeping Staff manage room maintenance activities, ensuring rooms are cleaned and prepared.
- **Generate Reports**: Admins have access to generate various reports such as occupancy rates, financial summaries, and other hotel performance metrics.

- **Payment**: Both Guests and Receptionists can interact with the payment process, handling the financial transactions related to reservations and services.

How It Works: Actors interact with different functionalities, each corresponding to their role. For example, a Guest is limited to making reservations, checking in/out, and making payments, while a receptionist has broader access.

System Responses: The "login error" use case shows that when users try to log in, the system includes a validation mechanism to manage incorrect login attempts.

Use Case Inclusion: The diagram uses the <<include>> notation to show that "login error" is part of the login process, meaning every login attempt can potentially lead to a success or an error.

Key Points: The diagram clearly separates the different roles (Admin, Receptionist, Guest, Housekeeping Staff) and links them to the functionalities they are responsible for. The system centralizes and automates the key operations for managing a hotel, providing specific use cases for each actor. Use cases are designed to reflect real-world interactions, ensuring all essential hotel operations are covered.

II.II Flowchart

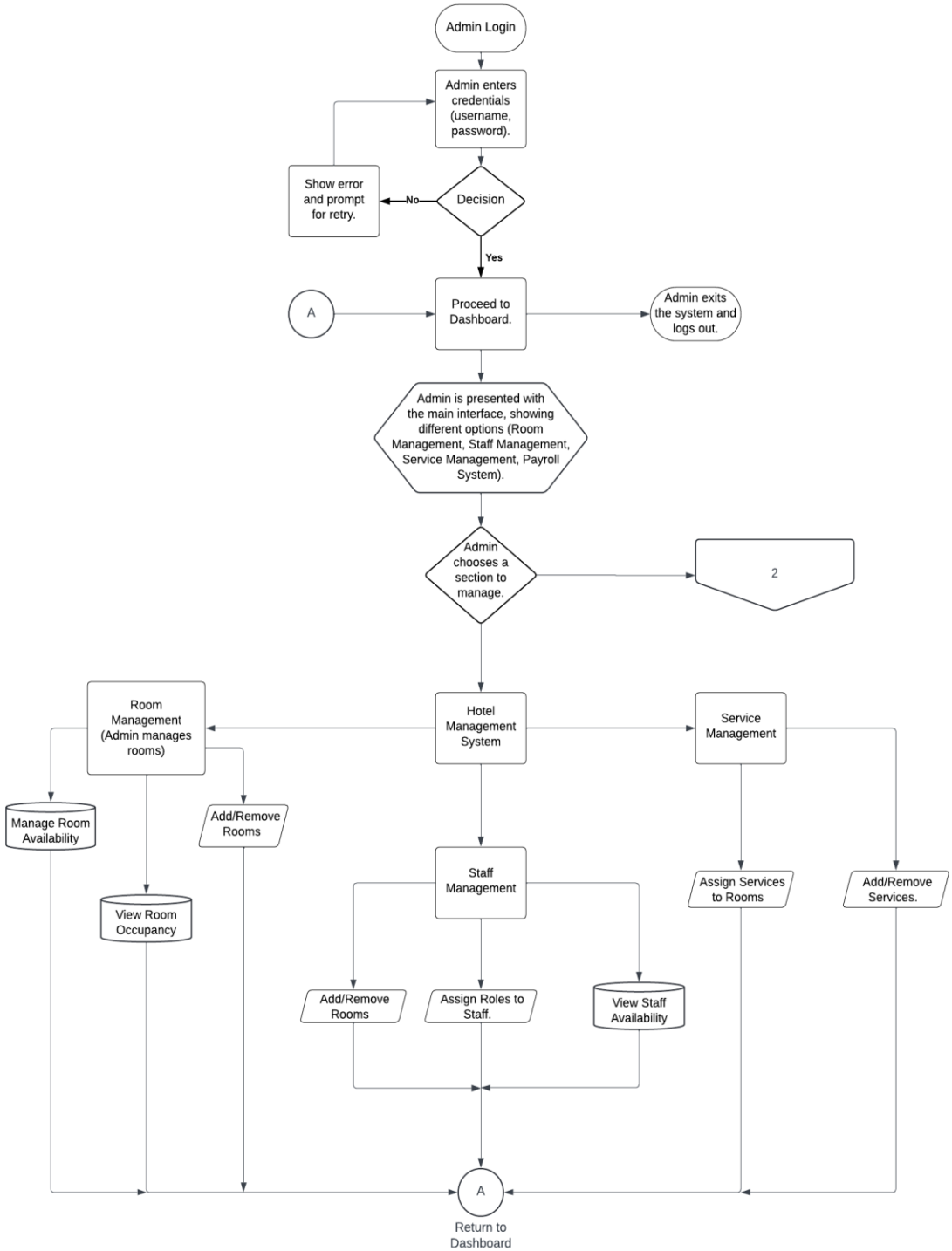

Fig 2.2.1 first half of our flow chart

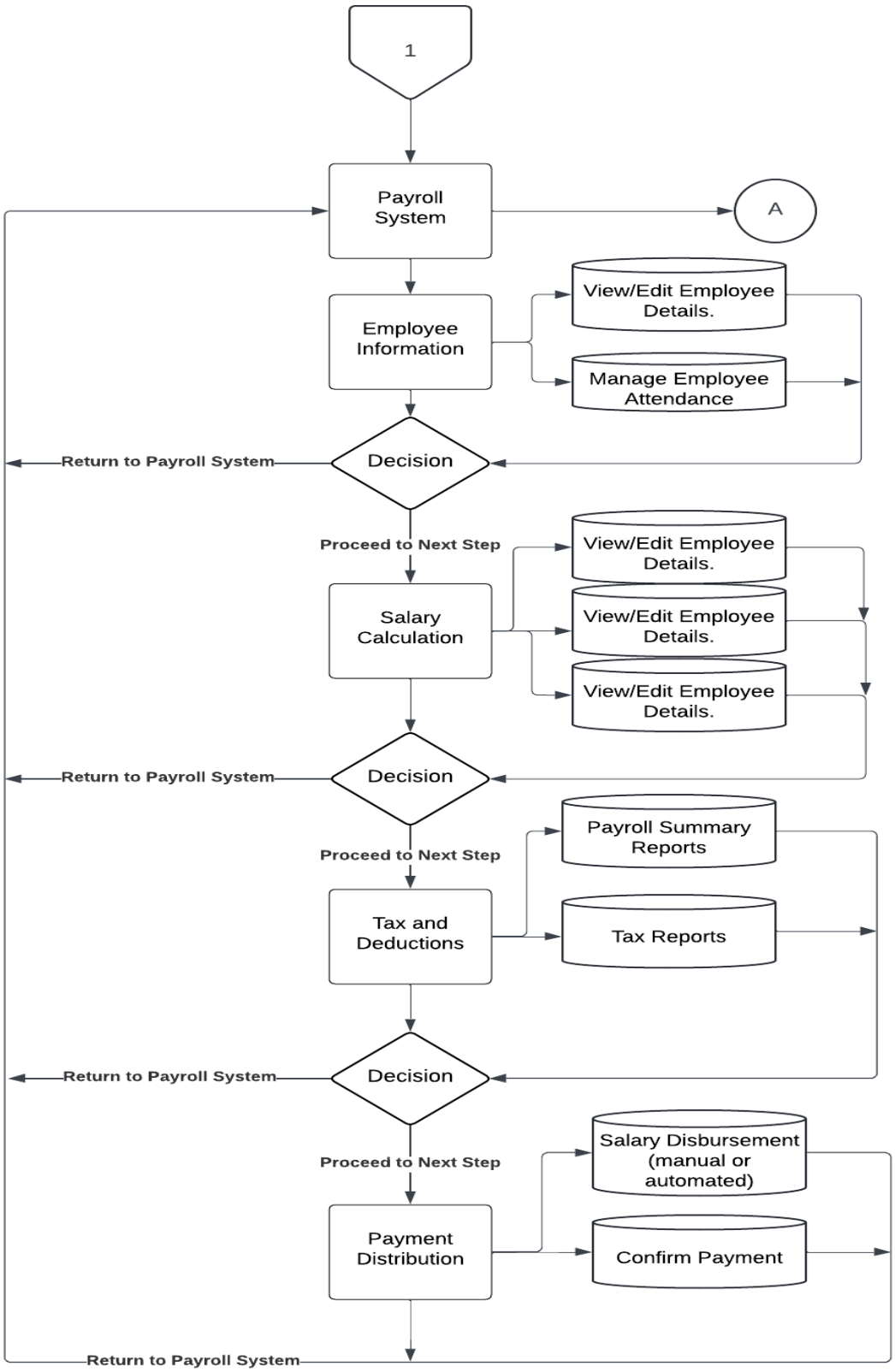

Fig 2.2.2 – second half of our flowchart

II.III Data Flow Diagram

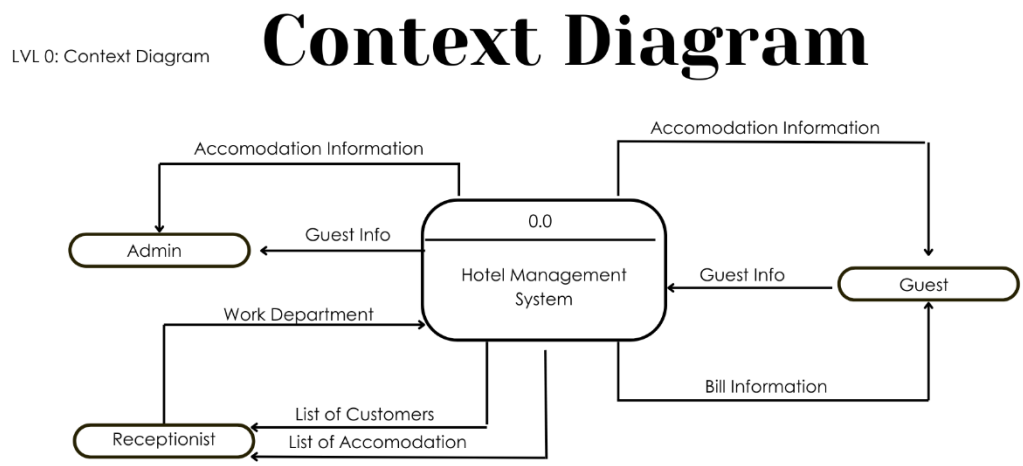

Figure 2.3.1 Level 0 Data Flow Diagram

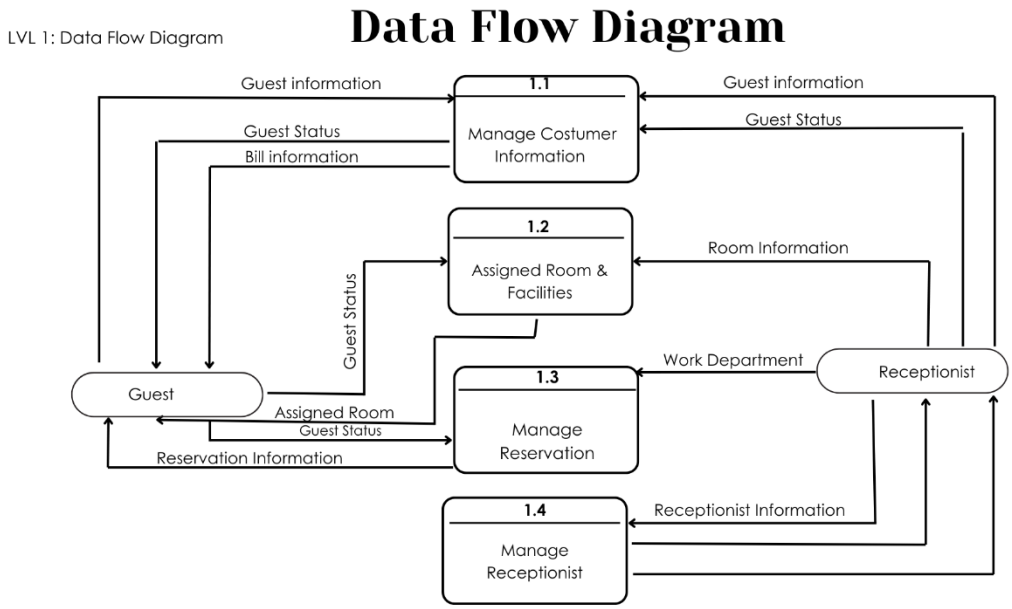

Figure 2.3.2 Level 1 Data Flow Diagram

II.IV Entity Relationship Diagram

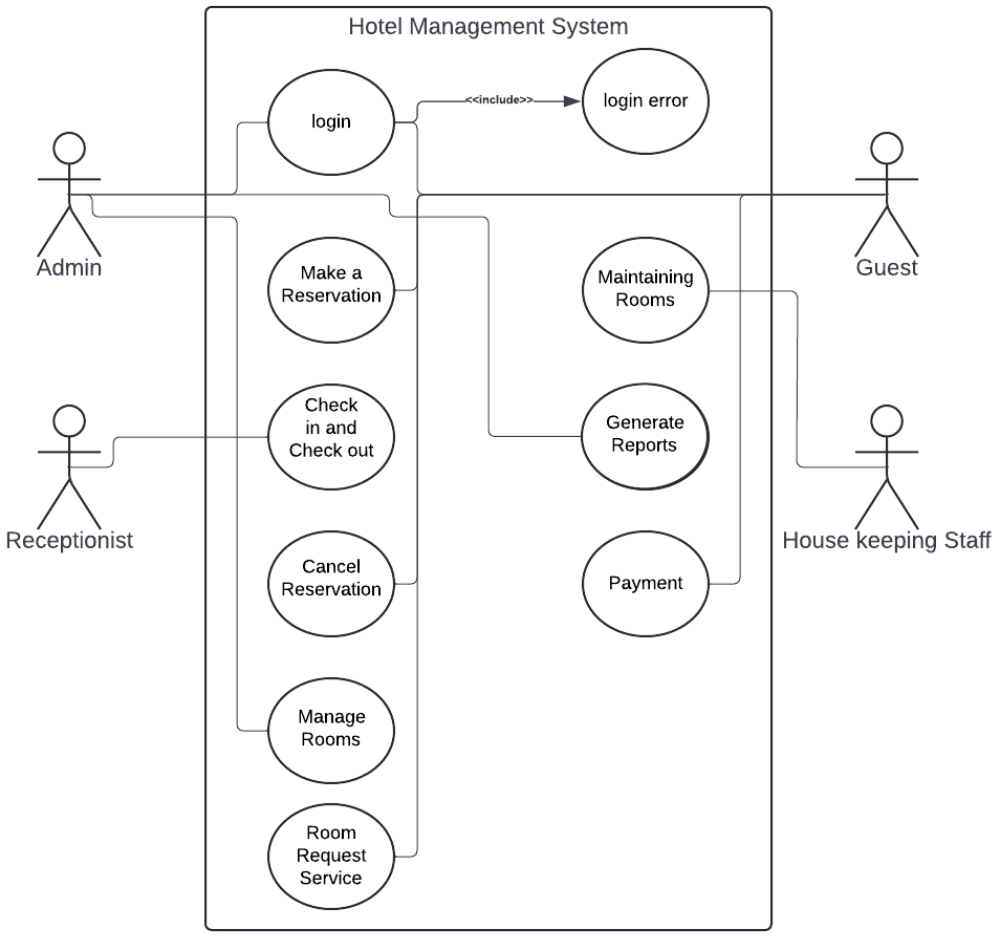

Figure 2.4.1 Entity Relationship Diagram

III. DEVELOPMENT TOOLS AND ENVIRONMENT

III.I PROGRAMMING LANGUAGE

C# Programming Language

III.II DEVELOPMENT TOOLS

- **Visual Studio:**

This will be the primary IDE for developing HotelEASE. It offers comprehensive support for C#, including built-in debugging tools, code refactoring, IntelliSense for faster coding, and easy integration with version control systems like Git. Visual Studio provides access to a range of libraries and frameworks in the .NET ecosystem, enhancing the efficiency of OOP-based development.

- **.NET Framework**

.NET Framework will be used to build the Windows-based application, providing essential APIs for OOP and seamless integration with databases, web services, and more.

- **Database Management: MySQL Server**

SQL Server will be used for managing the backend database, storing information related to guests, reservations, payroll, and other hotel data.

III.III METHODOLOGY

Our team has chosen Agile methodology for the hotel management system to ensure flexibility and adaptability throughout the development process. By working in short, iterative sprints, we can continuously gather feedback from hotel staff and management, allowing us to refine the system to meet their evolving needs. This approach enables us to deliver key features like room booking, guest services, and

a payroll for employees, ensuring quicker releases and a system that is closely aligned with user requirements.

IV. KEY SYSTEM COMPONENTS

IV.I ATTRIBUTES

1. **Admin Class**

Attributes:

adminID: Unique identifier for the admin.

name: Admin's full name.

email: Admin's contact email.

role: Specifies the admin's role (e.g., system admin, payroll admin).

Methods:

login(): Verifies the admin's login credentials.

generateReport(): Generates various reports (e.g. financial).

manageUsers(): Adds, updates, or removes user accounts (e.g. staff).

Object Interactions: The Admin interacts with the User class to manage system users. It interacts with Room and Reservation classes to generate reports on room occupancy and reservations. Admin also interacts with the Payroll class for employee payroll management.

2. **Employee Class (For Payroll Management)**

Attributes:

employeeID: Unique identifier for each employee.

name: Employee's full name.

position: The role of the employee (e.g., receptionist, housekeeping staff).

salary: The base salary or hourly wage for the employee.

workHours: The number of hours the employee has worked

payStatus: Indicates if the employee has been paid for the current pay period.

Methods:

clockIn(): Records the start of a work shift.

clockOut(): Records the end of a work shift.

calculatePay(): Calculates the salary or wages based on the employee's work hours or base salary.

Object Interactions: The Employee interacts with the Payroll class to receive payments and track working hours. Interacts with the Admin class for employee management (adding new employees, updating roles, etc.).

3. **Payroll Class**

Attributes:

payrollID: Unique ID for the payroll instance.

employeeList: List of employees included in this payroll run.

totalPayrollAmount: The total sum of salaries for all employees.

payPeriod: The date range for the payroll (e.g., monthly or weekly).

Methods:

processPayroll(): Processes payroll for all employees in the current pay period.

generatePayslip(employeeID): Generates a detailed payslip for each employee.

recordPayment(employeeID): Marks the employee as paid for the current pay period.

Object Interactions: Payroll interacts with Employee to retrieve salary and work hour data for processing payments. Admin interacts with Payroll to initiate the payroll process and view payroll reports.

4.Report Class

Attributes:

reportID: Unique ID for each report.

reportType: Type of report (e.g., financial, occupancy, payroll).

reportData: Data included in the report.

reportDate: Date when the report was generated.

Methods:

generateReport(): Generates specific reports based on the admin's request (e.g., financial reports, employee performance).

viewReport(): Allows admins to view and export reports.

Object Interactions: The Report class interacts with the Reservation, Room, Employee, and Payroll classes to gather the necessary data for generating detailed reports.

5.Room Class (for Admin management)

Attributes:

roomID: Unique ID for each room.

roomType: Type of room (e.g., single, double, suite).

status: Whether the room is available, booked, or under maintenance.

pricePerNight: The cost of the room per night.

Methods:

updateRoomStatus(): Updates the status of the room (e.g., available, booked).

viewRoomDetails(): Allows the admin to view room details (price, availability).

Object Interactions: The Room class interacts with Admin for report generation, room availability, and status updates.

Interaction Diagram: Admin and Payroll System

The admin interacts with the Employee class to manage employees, manage payroll, and generate payslips. They use the Report class to generate payroll-related reports and view detailed payment information, ensuring efficient system management and efficient employee management.

V - Advantages of Using OOP in the System (Admin and Payroll Perspective)

V.I Modularity:

Each class is designed to handle a specific responsibility, such as Admin for system management, Employee for employee details, and Payroll for salary processing. This separation makes the system modular and easier to maintain. For example, if changes are made to how payroll is calculated, you only need to modify the Payroll class, leaving the other parts of the system untouched.

V.II Flexibility:

The system is flexible, allowing easy modifications. For instance, new employee roles or payroll policies can be introduced by extending the Employee and Payroll classes. OOP's inheritance and polymorphism make it simple to add new functionalities without rewriting existing code.

VI - Potential Challenges and Solutions

VI.I - Challenges:

Increased Design Complexity: The inclusion of payroll and reporting features may introduce additional layers of complexity.

Overuse of Inheritance: Inheritance could be misused, leading to tight coupling between classes, making it hard to modify certain components (e.g., payroll calculation).

VI.II - Mitigation Strategies:

Design Patterns: Use Factory Patterns for object creation (e.g., creating different employee types) and Observer Patterns for notifying admins about payroll events.

Careful Analysis: Ensure inheritance is only used where necessary, and prefer composition for flexibility.

VII - Testing and Implementation Plan

VII.I - Unit Testing:

4. **Employee Class:** Test methods like clockIn(), clockOut(), and calculatePay() to ensure that the payroll system is accurate.
5. **Payroll Class:** Test processPayroll() and generatePayslip() to verify correct payment calculations.

VII.II - Integration Testing: Test interactions between the Admin, Employee, and Payroll classes to ensure that payroll processes work correctly, from data retrieval to payment processing.

VII.III - Deployment Strategy: Deploy payroll-related features in stages: Initial deployment of employee management (add/remove employees). Deployment of salary processing and payroll features. Final phase for reporting and viewing payslips.

VIII - Conclusion

VIII-I Summary:

The Admin side of HotelEASE is designed to manage users, generate reports, and handle payroll functions. Each class is modular and well-structured to ensure flexibility, allowing for easy updates and expansions.

VII-II Future Enhancements:

Automated Payroll Calculation based on dynamic work schedules.
AI Integration to suggest staffing requirements based on room occupancy trends. Cloud Integration for storing payroll data securely, allowing remote access by admins for reports and salary processing.

IX. - APPENDICES

IX.I Appendix A: Evidences of Collaboration

Meeting Minutes: 27

- Date: August 7, 2024 8:00pm
- Attendees: Dezire Barros, Jericho Martin Simon, Danilo Canlas JR, John Rey Cailo, Jheboy Larong
- **Key Points Discussed:**
    - Project timeline and deadlines
    - Responsibilities for each module (e.g., RFID module, payroll module)
    - Challenges faced in the development process
- **Action Items:**
    - **Jericho to finalize the database schema by September 16, 2024.**
    - **Dezire to implement user login functionality and update the team by August 20, 2024.**
- **Task Board Snapshot:**

- **Backlog:**
    - **User Login Implementation (Assigned to Dezire)**
    - **RFID Module Development (Assigned to Jericho)**
- **In Progress:**
    - **Payroll Integration (Assigned to Jheboy)**
- **Completed:**
    - **Database Schema Design (Assigned to Danilo)**

Collaborative Design Documents:

- **Document Title: *Hotel Ease Design Specifications***
- **Link: https://drive.google.com/drive/folders/1_A6KE_l6CqQWZWc3OGgcRxz3vY8sJZKQ?usp=drive_link**
- **Contributors:**
    - **Dezire Barros – Main Programmer**
    - **Jericho Martin Simon – User Interface Designer**
    - **Danilo Canlas JR - RFID Module Integration**
    - **John Rey Cailo – Payroll Programmer**
    - **Jheboy Larong – Payroll system UI Designer**

IX.II Appendix B : Results of Unit Testing and Integration

This appendix presents the detailed results of unit testing and integration testing for the HotelEASE system

Unit Testing

- **Objective:** To verify the functionality of individual components (e.g., classes, methods) within the system.
- **Testing Tools Used:** [Insert tools used, such as NUnit, JUnit, etc.]

Summary of Test Cases:

Test Case ID	Test Description	Input Data	Expected Output	Actual Output	Status (Pass/Fail)
UT_01	Test RFID card scanning functionality	Valid RFID card ID	Successfully read the card ID and return user information	User is logged in into the System and Database	Pass
UT_02	Test invalid RFID card handling	Invalid RFID card ID	Error message indicating invalid card ID	The card used in this is not registered yet.	Fail
UT_03	Test RFID card authorization	Valid RFID card ID	Access granted to user's account	Successfully timed-in and timed out the user	Pass
UT_04	Test RFID card deactivation	Valid RFID card ID	Successfully deactivate the card	Deactivated users RFID Card	Pass

Sample Results:

- **Test Case ID:** UT_01
- **Component:** RFID Card Reader
- **Description:** Test RFID card scanning functionality.

- **Input:** Valid RFID card ID.
- **Expected Result:** Successfully read the card ID and return user information.
- **Actual Result:**

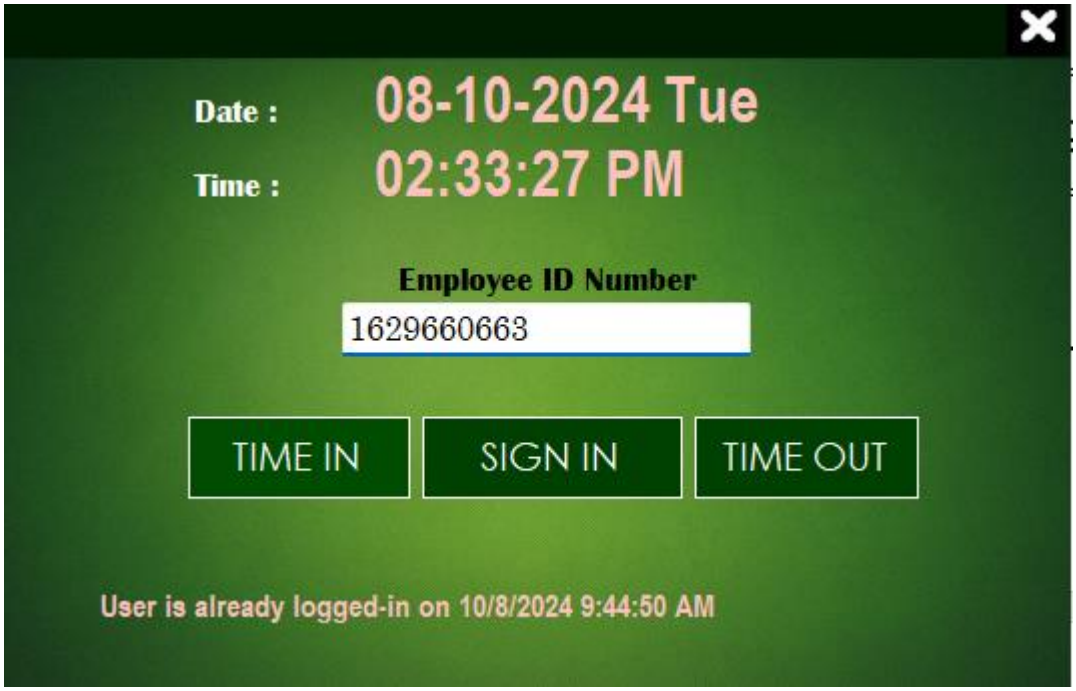

Fig.9.1 Time in Time Out interface

- **Status:** Pass

Conclusion: Overall, unit testing for the RFID functionalities showed 90% in 9 out of 10 tests. The failed test case was due to double tapping of the card and double recording into the system Recommendations for improvement include additional UI changes into the time in time out system to improve the visibility of the time in/time out notice

IX.III Appendix C: Curriculum Vitae

// Please see the next page for our individual Curriculum Vitae